УГНТУ • ЦЕНТР ТОП ИТ • 09.03.03 ПРИКЛАДНАЯ ИНФОРМАТИКА

Программирование и прототипирование ИТ-решений

Модульность и управление зависимостями модули, пакеты, pip, виртуальные окружения

ЛЕКЦИЯ 10

Ахметов Ильнур Вазирович

План лекции

1. Импорт стандартных модулей и создание собственных (пространства имён).
2. Пакетная структура Python-проекта (роль __init__.py).
3. Менеджер пакетов pip и фиксация зависимостей (requirements.txt).
4. Изоляция зависимостей: виртуальные окружения (venv).
5. Ловушки: ModuleNotFoundError, круговые импорты, не то окружение.
6. Мини-проект: превращаем скрипты лабораторной в проект с пакетами.

В конце — вопросы для закрепления (по одному на слайд).

ПОВТОРЕНИЕ

Что мы уже умеем

- Лекция 8: словари и множества, выбор структуры.
- Лекция 9: функции — def, аргументы, LEGB, lambda.
- Мы писали весь код в ОДНОМ файле, одним блоком.
- Настоящие проекты: десятки файлов, чужие библиотеки,
- разные версии пакетов в разных проектах.
- Сегодня: как код разбить на модули и пакеты,
- как подключать чужие библиотеки (pip)
- и как изолировать проекты друг от друга (venv).

```
что нас ждёт
# сегодня в консоли и терминале:

>>> import rates        # свой модуль
>>> rates.oil_rate(120.0, 0.35)

PS> python -m venv .venv
PS> .venv\Scripts\Activate
(.venv) PS> pip install pandas
(.venv) PS> pip freeze > requirements.txt
```

Зачем нужны модули

- Скрипт растёт: 500 строк в одном файле — читать больно.
- Модуль — файл .py: свой набор функций и имён.
- Разделение ответственности: расчёты / загрузка / отчёт.
- Переиспользование: rates.py подключается в любой проект.
- Пространство имён: rates.oil_rate не конфликтует
- с вашей oil_rate в другом файле.
- Модуль — единица ОРГАНИЗАЦИИ кода, как функция — поведения.

```
один файл против модулей
# было: analysis.py, всё в одном
def load(): ...
def oil_rate(): ...
def report(): ...
load(); report()

# стало: три файла-модуля
# data_io.py  -> load(), save()
# rates.py    -> oil_rate()
# main.py     -> собирает всё вместе
```

import: три формы

- import math — весь модуль, доступ через math.sqrt(2).
- from math import sqrt — конкретное имя, доступ напрямую.
- import numpy as np — псевдоним: короче и привычнее.
- Правило: import модуль — умолчание;
- from ... import — когда имя нужно часто.
- from math import * — ЗАПРЕЩЕНО: заливает имён,
- непонятно, чей sqrt — ваш или модуля.
- Каждая форма создаёт разный набор имён у вас в файле.

консоль Python

```
>>> import math
>>> math.sqrt(16)
4.0

>>> from statistics import mean
>>> mean([1, 2, 3])
2

>>> import numpy as np
>>> np.mean([1, 2, 3])
2.0
```

Стандартная библиотека

- ~200 модулей УЖЕ установлены вместе с Python.
- Ничего ставить не надо: import и работайте.
- math — функции и константы; statistics — mean, median.
- datetime — даты и время; json — чтение/запись JSON.
- csv — таблицы; pathlib — пути к файлам.
- random — случайность; itertools — комбинаторика;
- functools — reduce, partial, lru_cache (лекция 9!).
- Прежде чем ставить чужое — проверьте стандартную.

полезные модули

```
>>> import statistics as st
>>> st.mean([85.4, 12.7, 65.3])
54.46666666666667

>>> from datetime import date
>>> date.today()
datetime.date(2025, 11, 3)

>>> import json
>>> json.dumps({'wct': 0.35})
'{"wct": 0.35}'
```

Свой модуль

- Любой .py-файл — модуль: rates.py = модуль rates.
- import rates — выполняет файл один раз
- и даёт доступ к его именам: rates.oil_rate(...).
- Требования: файл лежит РЯДОМ (в sys.path),
- имя — латиницей, без дефисов, не как у библиотек.
- Свой модуль ничем не хуже встроенного:
- те же import, те же точки, тот же help().

rates.py + main.py

```
# rates.py
def oil_rate(q_liq, wct=0.0):
    """Дебит нефти."""
    return round(q_liq * (1 - wct), 2)

# main.py (лежит рядом)
import rates
print(rates.oil_rate(120.0, 0.35))
78.0
```

Пространства имён

- У КАЖДОГО модуля — своя область глобальных имён.
- rates.oil_rate — «oil_rate из модуля rates».
- Одноимённые функции в разных модулях не конфликтуют:
- rates.oil_rate и main.oil_rate — разные имена.
- Модуль = самостоятельный «словарь» имён (лекция 8).
- dir(rates) — все имена модуля;
- rates.__name__ — строка 'rates'.
- Это та же изоляция, что у функций (LEGB), но на уровне файлов.

консоль Python

```
>>> import rates
>>> rates.__name__
'rates'

>>> def oil_rate(q): return q   # своя
>>> oil_rate(120.0)
120.0
>>> rates.oil_rate(120.0, 0.35)
78.0

>>> dir(rates)[:4]
['__builtins__', '__doc__', ...]
```

Модуль выполняется один раз

- Первый import выполняет файл модуля ЦЕЛИКОМ.
- Результат кладётся в кэш sys.modules.
- Повторные import берут из кэша — файл НЕ выполняется.
- Следствие 1: «тяжёлый» импорт платим один раз.
- Следствие 2: правите модуль — нужен ПЕРЕЗАПУСК
- скрипта (или ядра в Jupyter!).
- sys.modules — словарь «имя → объект модуля»;
- импорт — в первую очередь поиск по этому словарю.

кэш импортов

```
# rates.py
print('модуль rates загружается')
def oil_rate(q, w=0.0): ...

>>> import rates
модуль rates загружается
>>> import rates
>>>            # тишина: взят из кэша

>>> import sys
>>> 'rates' in sys.modules
True
```

if __name__ == "__main__"

- Двойная жизнь модуля: библиотека ИЛИ скрипт.
- При запуске файла напрямую __name__ == '__main__'.
- При import тем же файлом __name__ == 'rates'.
- Код под if — выполняется только при запуске:
- демо, тесты, main() — не мешают импортеру.
- Стандартная концовка каждого модуля проекта.
- Запомните дословно: это идиома Python.

```
rates.py
def oil_rate(q_liq, wct=0.0):
    return round(q_liq * (1 - wct), 2)

if __name__ == '__main__':
    # только при запуске:
    # python rates.py
    print(oil_rate(120.0, 0.35))

>>> import rates   # print НЕ сработал
>>> rates.__name__
'rates'
```

Как Python ищет модули

- import rates — где Python берёт файл rates.py?
- sys.path — список папок поиска, ПО ПОРЯДКУ:
- 1) папка запускаемого скрипта;
- 2) папки стандартной библиотеки;
- 3) site-packages — чужие пакеты (pip!).
- Нашёл первый — остальное игнорирует:
- одноимённый свой файл ЗАТЕНЯЕТ библиотеку.
- ModuleNotFoundError = имени нет нигде в path.

sys.path

```
>>> import sys
>>> sys.path[:3]
['C:\\proj',
 'C:\\Python312\\python312.zip',
 'C:\\Python312\\Lib']

# свой rates.py в папке проекта
# затенит чужой rates, если был

>>> import rates
>>> rates.__file__   # где нашли файл
'C:\\proj\\rates.py'
```

from-import: осторожно

- from rates import oil_rate — имя без префикса.
- Удобно, если вызываете 10 раз в строках подряд.
- Опасность: oil_rate «сваливается» в ваш файл;
- читатель не видит, откуда имя (свой? чужой?).
- from rates import * — все публичные имена сразу:
- затенение, нечитаемость — так не делаем.
- Компромисс: from rates import oil_rate as rate_oil.
- Правило: import модуль — умолчание, from — осознанно.

```
читаемость против краткости
# читателю понятно:
import rates
rates.oil_rate(120.0, 0.35)

# короче, но откуда oil_rate?
from rates import oil_rate
oil_rate(120.0, 0.35)

# ЗАПРЕЩЕНО:
from rates import *
oil_rate(120.0, 0.35)   # чья функция?
```

Импорт внутри функции

- import — обычный оператор: работает где угодно,
- в том числе ВНУТРИ функции или ветки if.
- Ленивый импорт: тяжёлый модуль грузится,
- только если функция реально вызвана.
- Применение: pandas нужен лишь для одного отчёта —
- не тянем его при обычном запуске.
- Правило стиля: импорты — вверху файла (PEP 8);
- ленивый импорт — осознанная оптимизация.

ленивый импорт

```
def export_excel(df):
    import pandas as pd   # только тут
    df.to_excel('report.xlsx')
    return True

# без вызова export_excel()
# pandas не загружается вовсе:
>>> import main
>>> 'pandas' in sys.modules
False
```

Свои и чужие модули

- Три источника модулей:
- 1) стандартная библиотека — уже есть;
- 2) свои .py-файлы — лежат в проекте;
- 3) сторонние пакеты — ставятся через pip
- в site-packages (pandas, numpy, openpyxl).
- Для Python все три одинаковы: import имя.
- Разница — в установке и обновлении:
- свои правим сами, чужие ставит pip.

```
три источника
import math          # стандартная
import rates         # свой файл
import pandas        # сторонний (pip)

# у всех одинаковая механика:
>>> math.__file__ is not None
True
>>> rates.__file__
'C:\\proj\\rates.py'
>>> import pandas
>>> pandas.__file__   # site-packages
```

Практикум: модуль rates.py

- Собираем модуль из лекции 9:
- oil_rate, валовый расчёт, mean.
- Docstring у каждой функции (контракт!).
- Гварды на входах (лекция 9).
- Блок if __name__ == '__main__' — демо.
- Импортируем из main.py и считаем дебит куста.
- Проверяем help(rates) — документация на месте.

```
rates.py целиком
"""Расчёты дебита нефти."""

def oil_rate(q_liq, wct=0.0):
    """Дебит нефти: q_liq*(1-wct)."""
    if q_liq <= 0: return 0.0
    return round(q_liq * (1 - wct), 2)

def bush_oil(rows):
    """Суммарный дебит куста."""
    return round(sum(oil_rate(q, w)
        for _, q, w in rows), 2)

if __name__ == '__main__':
    print(oil_rate(120.0, 0.35))
```

Пакет: папка модулей

- Проект растёт: 10 модулей в одной папке — каша.
- Пакет — ПАПКА с модулями (и подпакетами).
- oil/ -> rates.py, data_io.py, report.py.
- Импорт через точку: from oil import rates.
- Точка = «спуститься в папку».
- Иерархия имён: oil.rates.oil_rate —
- полный адрес функции в структуре проекта.
- Пакеты группируют по смыслу, модули — по теме.

```
структура
proj/
  oil/                # пакет
    __init__.py
    rates.py
    data_io.py
    report.py
  main.py

# main.py:
from oil import rates
rates.oil_rate(120.0, 0.35)
```

__init__.py: роль

- __init__.py — файл-маркер: «эта папка — пакет».
- Выполняется при ПЕРВОМ импорте пакета.
- Может быть пустым — этого достаточно.
- Может экспортировать API: from .rates import oil_rate
- — тогда пользователь пишет from oil import oil_rate.
- «Внутренности скрыты, наружу — только контракт».
- Без __init__.py папка — просто папка
- (утиная «неявная сборка» нам не нужна).

публичный API

```
# oil/__init__.py
from .rates import oil_rate, bush_oil
from .data_io import load_rows

__all__ = ['oil_rate', 'bush_oil',
           'load_rows']

# теперь коротко:
from oil import oil_rate
oil_rate(120.0, 0.35)   # 78.0
```

Импорт из пакета

- from oil import rates — модуль целиком.
- from oil.rates import oil_rate — функция сразу.
- import oil.rates — длинный префикс, редко нужно.
- from oil import oil_rate — через __init__ (слайд 2).
- Что выбрать: то, что читается яснее всего.
- Глубина больше двух уровней — сигнал,
- что структура слишком хитрая.
- Имя пакета — как имя модуля: коротко, латиницей.

варианты импорта

```
from oil import rates
rates.oil_rate(120.0, 0.35)

from oil.rates import oil_rate
oil_rate(120.0, 0.35)

import oil.rates
oil.rates.oil_rate(120.0, 0.35)

from oil import oil_rate  # через __init__
```

Структура проекта

- Типовой учебный проект анализа данных:
- proj/ — корень; main.py — вход;
- proj/oil/ — пакет с логикой;
- proj/data/ — сырые данные (не код!);
- proj/out/ — результаты, отчёты.
- Отделяем: код / данные / результаты.
- main.py — тонкий: загрузить → посчитать → сохранить.
- Вся логика — в пакете, main только дирижирует.

дерево проекта

```
proj/
  main.py            # вход
  oil/
    __init__.py
    rates.py         # расчёты
    data_io.py       # load/save
    report.py        # отчёты
  data/
    wells.csv
  out/
    report.xlsx
```

Вложенные пакеты

- Внутри пакета — подпакеты: oil/importers/, oil/exports/.
- Каждая вложенная папка — свой __init__.py.
- import oil.importers.csv_loader — полный адрес.
- Зачем: версии загрузчиков под разные форматы
- (csv, excel, база) не мешают друг другу.
- Правило: вкладывать глубже двух уровней —
- только при реальной необходимости.
- Плоская структура читается лучше глубокой.

подпакеты

```
oil/
  __init__.py
  rates.py
  importers/
    __init__.py
    csv_loader.py
    excel_loader.py

from oil.importers.csv_loader import load_csv
rows = load_csv('data/wells.csv')
```

Относительные импорты

- Внутри пакета модули ссылаются друг на друга:
- from .rates import oil_rate — «сосед из моего пакета».
- from . import rates — весь соседний модуль.
- from .. import helpers — уровень выше (редко).
- Точка = «от текущего пакета», две — «от родителя».
- Работают ТОЛЬКО внутри пакета,
- не в запускаемом напрямую скрипте.
- Смешение относительных и абсолютных — источник путаницы:
- в учебных проектах — абсолютные импорты.

внутри oil/report.py

```
# абсолютный (умолчание):
from oil.rates import oil_rate

# относительный:
from .rates import oil_rate

# тоже относительный:
from . import data_io
rows = data_io.load_rows('data/wells.csv')
report = [oil_rate(q, w) for _, q, w in rows]
```

__all__ и публичный API

- Пакет — библиотека для своих же скриптов.
- Что наружу — решаете вы: контракт пакета.
- __all__ = [...] — список публичных имён:
- управляет from oil import * (и документацией).
- Префикс _имя — «внутреннее, не трогайте».
- Маленький API лучше большого:
- сколько функций реально используют другие модули?
- Остальное — внутренняя кухня.

контракт пакета

```
# oil/__init__.py
from .rates import oil_rate, bush_oil
from .data_io import load_rows

__all__ = ['oil_rate', 'bush_oil',
           'load_rows']

# rates.py: _norm() - внутренняя
def _norm(w): return min(max(w, 0), 1)
def oil_rate(q, w=0.0):
    return round(q * (1 - _norm(w)), 2)
```

Реальный проект: как устроен

- Взглянем на зрелый проект (структура типовая):
- src/ — код пакета (src-layout);
- tests/ — тесты (импорт пакета как библиотеки);
- README.md — что это и как запустить;
- requirements.txt — зависимости (следующий блок!);
- pyproject.toml — метаданные сборки.
- main.py и скрипты — тонкие надстройки над пакетом.
- Учебный проект — та же форма, меньше файлов.

типовое дерево

```
oil_proj/
  src/
    oil/
      __init__.py
      rates.py
  tests/
    test_rates.py
  README.md
  requirements.txt
  pyproject.toml
  .venv/              # не в git!
```

Практикум: скрипты → пакет

- Превращаем лабораторную в проект:
- 1) создать папку oil/, добавить __init__.py;
- 2) перенести rates.py, data_io.py внутрь;
- 3) починить импорты (from oil.rates import ...);
- 4) main.py оставить снаружи, сделать тонким;
- 5) запустить python main.py — всё работает?
- Критерий успеха: python -c "import oil" без ошибок
- и запуск main.py из корня.

main.py после рефакторинга

```
from oil import load_rows, bush_oil

def main():
    rows = load_rows('data/wells.csv')
    total = bush_oil(rows)
    print(f'Дебит куста: {total} т/сут')

if __name__ == '__main__':
    main()

# PS> python main.py
# Дебит куста: 118.0 т/сут
```

PyPI и сторонние пакеты

- Стандартной библиотеки не всегда хватает:
- pandas — таблицы, openpyxl — Excel,
- matplotlib — графики, requests — HTTP.
- PyPI (pypi.org) — витрина: >500 000 пакетов.
- pip — инструмент установки оттуда.
- pip ставит пакет в site-packages текущего Python.
- После установки: import pandas — как родной.
- Аналогия: PyPI — магазин, pip — доставка.

терминал

```
PS> pip install pandas
Collecting pandas
  Downloading pandas-2.2.3-cp312-...
Installing collected packages: pandas
Successfully installed pandas-2.2.3

>>> import pandas as pd
>>> pd.__version__
'2.2.3'
```

pip install: что происходит

- 1) pip находит пакет на PyPI по имени.
- 2) скачивает wheel (.whl) — собранный архив.
- 3) распаковывает в site-packages.
- 4) ставит ЗАВИСИМОСТИ пакета (numpy, dateutil...).
- 5) пишет метаданные: имя, версия, файлы.
- Итого: import работает через sys.path
- (site-packages в нём, слайд «поиск модулей»).
- Удаление: pip uninstall pandas.

терминал

```
PS> pip install openpyxl
Collecting openpyxl
  Downloading openpyxl-3.1.5-py2.py3-none-...
Installing collected packages: openpyxl
Successfully installed openpyxl-3.1.5

PS> pip uninstall openpyxl
Proceed (Y/n)? y
Successfully uninstalled openpyxl-3.1.5
```

Версии пакетов

- Пакеты живут версиями: 2.2.3 (мажор.минор.патч).
- pandas==2.2.3 — ровно эта версия.
- pandas>=2.0 — не ниже второй.
- pandas~=2.2 — «совместимо»: 2.2.x, не 3.0.
- Без указания pip ставит самую свежую —
- а она может ломать ваш код (другой API).
- Воспроизводимость = фиксировать версии.
- Принцип: сегодня работает — должно работать и завтра.

спецификаторы версий

```
PS> pip install pandas==2.2.3
Successfully installed pandas-2.2.3

PS> pip install 'pandas>=2.0'
# поставит последнюю из 2.x или 3.x

PS> pip install 'pandas~=2.2'
# 2.2.x - без перехода на 3.0

>>> pd.__version__
'2.2.3'
```

requirements.txt

- requirements.txt — список зависимостей проекта:
- по пакету на строку, с версиями.
- Это КОНТРАКТ проекта: «что нужно поставить».
- Новый разработчик (или вы на новой машине):
- pip install -r requirements.txt — и всё работает.
- Файл лежит в корне проекта и хранится в git.
- Без него: «у меня работало» — не воспроизводится.
- Имя файла — договорённость всей экосистемы.

requirements.txt

```
# requirements.txt
pandas==2.2.3
openpyxl==3.1.5
matplotlib==3.9.2

# установка одной командой:
PS> pip install -r requirements.txt
Successfully installed pandas-2.2.3
  openpyxl-3.1.5 matplotlib-3.9.2
```

pip freeze: фиксация

- pip freeze — точный список УСТАНОВЛЕННОГО:
- все пакеты с точными версиями (==).
- pip freeze > requirements.txt —
- снимок окружения в файл.
- Порядок рабочий: поставили → проверили →
- зафиксировали.
- freeze пишет ВСЁ, включая транзитивные зависимости
- (иногда их десятки — это нормально).

терминал

```
PS> pip freeze
numpy==2.1.3
openpyxl==3.1.5
pandas==2.2.3
python-dateutil==2.9.0.post0

PS> pip freeze > requirements.txt
PS> type requirements.txt
numpy==2.1.3
openpyxl==3.1.5
...
```

pip list / pip show

- pip list — таблица «пакет — версия»
- (читабельнее freeze, без ==).
- pip show pandas — паспорт пакета:
- версия, автор, домашняя страница, зависимости.
- pip list --outdated — что устарело.
- pip install --upgrade pandas — обновить.
- Это ежедневные инструменты диагностики:
- «что стоит? какая версия? от кого зависит?»

```
паспорт пакета
PS> pip show pandas
Name: pandas
Version: 2.2.3
Summary: Powerful data structures...
Requires: numpy, python-dateutil, tzdata
Required-by: -

PS> pip list
Package    Version
---------- -------
numpy      2.1.3
pandas     2.2.3
```

Транзитивные зависимости

- pandas не работает один: ему нужны
- numpy, python-dateutil, tzdata...
- Это транзитивные зависимости —
- зависимости ваших зависимостей.
- pip ставит их автоматически (рекурсивно).
- Дерево: ваш проект → pandas → numpy.
- pip show pandas → Requires: ... —
- посмотреть дерево одного уровня.
- Итого ставится гораздо больше, чем просили.

```
дерево зависимостей
ваш проект
├── pandas==2.2.3
│   ├── numpy==2.1.3
│   ├── python-dateutil==2.9.0
│   └── tzdata==2024.2
└── openpyxl==3.1.5
    └── et-xmlfile==2.0.0

PS> pip show pandas | findstr Requires
Requires: numpy, python-dateutil, tzdata
```

Конфликты версий

- Проект A хочет pandas==2.0, проект B — 3.0.
- В ОДНОМ окружении обе версии не живут:
- пакет = одна папка в site-packages.
- pip предупреждает о конфликте требований,
- иногда ломая уже установленное.
- Решение — изоляция окружений (блок venv!):
- каждому проекту — свой site-packages.
- Также конфликты внутри одного набора:
- pip resolver пересчитывает дерево при каждой установке.

конфликт в терминале

```
PS> pip install pandas==2.0.0
ERROR: pip's dependency resolver...
pandas 2.0.0 requires numpy<1.25,>=1.20.3,
but you have numpy 2.1.3 which is
incompatible.

# вывод: каждому проекту - свой venv
# (следующий блок лекции)
```

Правила работы с зависимостями

- 1) Ставим минимум: каждый пакет — риск и вес.
- 2) Версии фиксируем (== или ~=).
- 3) requirements.txt обновляем при каждой смене.
- 4) Обновляем осознанно: прочитать changelog,
- прогнать проект, только потом freeze.
- 5) Никогда не правим файлы в site-packages!
- 6) requirements.txt — в git; .venv — НЕТ.
- Мир инструментария шире: poetry, uv, conda —
- но pip + venv — база, которую надо знать всем.

чек-лист

```
# новый проект - 6 команд:
PS> python -m venv .venv
PS> .venv\Scripts\Activate
(.venv) PS> pip install pandas
(.venv) PS> pip freeze > requirements.txt
(.venv) PS> git add requirements.txt

# воспроизведение на новой машине:
PS> pip install -r requirements.txt
```

Проблема: глобальный Python

- По умолчанию pip ставит в ОДИН site-packages
- «системного» Python — общий на все проекты.
- Проект А: pandas 2.0. Проект Б: pandas 3.0.
- Обновили ради Б — сломался А. Откатили — Б.
- Через год: сотня пакетов, никто не знает,
- какой проект что использует.
- «Works on my machine» — источник этой шутки.
- Нужно: каждому проекту — своя песочница.

```
боль: один site-packages
C:\Python312\Lib\site-packages\
  pandas\        # чья версия?
  numpy\
  openpyxl\
  ... 130 пакетов ...

PS> pip install pandas==3.0.0
# проект на 2.x - сломан
# молча
```

venv: что это

- venv — встроенный механизм изоляции
- (модуль standard library, с версии 3.3).
- Идея: в проекте лежит папка .venv/
- с КОПИЕЙ интерпретатора и своим site-packages.
- python -m venv .venv — создать один раз.
- Активировал окружение → pip ставит В НЕГО,
- python запускает ЕГО интерпретатор.
- Снаружи .venv никто не трогает.

создание

```
PS> cd C:\proj
PS> python -m venv .venv

proj/
  .venv\                # окружение
    Scripts\python.exe
    Lib\site-packages\
  main.py
  requirements.txt
```

Активация: Windows

- Активация = переключить терминал на окружение.
- PowerShell: .venv\Scripts\Activate.ps1
- cmd: .venv\Scripts\activate.bat
- Признак: префикс (.venv) в строке приглашения.
- Деактивация: deactivate — вернуться к системному.
- Если PowerShell блокирует скрипты —
- Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
- (один раз, это политика запуска скриптов).

терминал

```
PS> .venv\Scripts\Activate.ps1
(.venv) PS>

(.venv) PS> python -c "import sys;
  print(sys.prefix)"
C:\proj\.venv

(.venv) PS> deactivate
PS>
```

Активация: Linux и macOS

- Те же шаги, другой путь активации:
- source .venv/bin/activate
- (в Windows — Scripts/, здесь — bin/).
- Признак тот же: (.venv) в приглашении.
- Деактивация: deactivate.
- Зачем знать обе платформы:
- серверы почти всегда Linux,
- а ноутбуки коллег — что угодно.

```bash
$ python3 -m venv .venv
$ source .venv/bin/activate
(.venv) $ python -c "import sys;
  print(sys.prefix)"
/home/user/proj/.venv

(.venv) $ deactivate
$
```

Что внутри .venv

- .venv/Scripts (bin) — python.exe, pip.exe, activate.
- .venv/Lib/site-packages — сюда pip ставит пакеты.
- pyvenv.cfg — конфиг: где базовый Python.
- Интерпретатор здесь — «ссылка» на системный,
- но с ПЕРЕНАПРАВЛЕННЫМ site-packages.
- Проверка: sys.prefix и sys.executable
- после активации показывают .venv.
- Удалить окружение = удалить папку. Безопасно.

```
анатомия
proj/.venv/
  pyvenv.cfg
  Scripts/
    python.exe
    pip.exe
    Activate.ps1
  Lib/
    site-packages/
      pandas\

>>> import sys; sys.executable
'C:\\proj\\.venv\\Scripts\\python.exe'
```

pip внутри venv

- Активное окружение перенаправляет pip:
- пакеты падают в .venv\Lib\site-packages.
- pip list ДО активации — системный список,
- ПОСЛЕ — только то, что в проекте.
- Проблема «каждый раз activate» —
- решается выбором интерпретатора в IDE (слайд 8).
- Проверка, что вы не промахнулись:
- where python (win) / which python (linux).

```
до и после
PS> pip list
# 130 пакетов системного Python

PS> .venv\Scripts\Activate.ps1
(.venv) PS> pip list
Package    Version
---------- -------
pip        24.2

(.venv) PS> pip install pandas
# упало в .venv, не в систему
```

Полный цикл проекта

- Чек-лист нового проекта — запишите:
- 1) mkdir proj && cd proj;
- 2) python -m venv .venv;
- 3) .venv\Scripts\Activate;
- 4) pip install нужного;
- 5) pip freeze > requirements.txt;
- 6) кодим; при смене зависимостей — обновить файл.
- Воспроизведение на любой машине:
- venv + pip install -r requirements.txt.

```
терминал: от нуля до работы
PS> mkdir oil_proj; cd oil_proj
PS> python -m venv .venv
PS> .venv\Scripts\Activate.ps1
(.venv) PS> pip install pandas openpyxl
(.venv) PS> pip freeze > requirements.txt
(.venv) PS> python main.py
Дебит куста: 118.0 т/сут

# коллега на новой машине:
PS> pip install -r requirements.txt
```

venv и IDE

- VS Code: Ctrl+Shift+P → «Python: Select
- Interpreter» → выбрать .venv\Scripts\python.exe.
- IDE сама активирует окружение в терминале
- и подсказывает имена из ЕГО пакетов.
- PyCharm: Settings → Project → Interpreter.
- Jupyter: выбор ядра (kernel) = выбор окружения.
- Признак верной настройки: автодополнение pandas
- работает, а импорты не подсвечены красным.

```
VS Code
Ctrl+Shift+P
> Python: Select Interpreter
  Enter/Select Interpreter...

  C:\proj\.venv\Scripts\python.exe
  ('.venv': venv)      <- выбрать

# внизу справа в статус-баре:
# 3.12 (.venv)
```

venv в команде

- В git хранится: код, requirements.txt, README.
- НЕ хранится: .venv (это бинарники под вашу ОС),
- data/ больших размеров, out/ результатов.
- .gitignore: .venv/, __pycache__/, out/.
- Правило обмена: «склонируй и сделай
- pip install -r requirements.txt».
- Окружение пересоздаётся за минуту —
- поэтому его и не хранят.
- Несколько проектов = несколько .venv,
- по одному в папке каждого проекта.

```
.gitignore
# .gitignore
.venv/
__pycache__/
out/
data/*.csv

# что видит коллега после clone:
proj/
  oil/
  main.py
  requirements.txt
  README.md
```

ModuleNotFoundError

- ModuleNotFoundError: No module named 'rates'.
- Причины по частоте:
- 1) файла нет / не в той папке (sys.path);
- 2) запустили не из корня проекта;
- 3) пакет не установлен в ЭТО окружение;
- 4) опечатка в имени (Rates, rate).
- Диагностика: print(sys.path) — где ищем;
- pip list — что стоит; __file__ соседа — откуда взяли.

диагностика

```
>>> import ratez
ModuleNotFoundError: No module named 'ratez'

# три вопроса:
# 1. как точно называется модуль?
# 2. откуда запущен скрипт?
>>> import sys; print(sys.path[0])
'C:\\proj\\oil'   # а rates.py выше!
# 3. тот ли Python? (.venv активен?)
```

Круговые импорты

- rates.py: from .report import build
- report.py: from .rates import oil_rate
- Каждый тянет другого — круг замкнулся.
- Симптом: ImportError / partially initialized
- module ... (most likely due to a circular import).
- Причина: модули спроектированы зависимыми
- в обе стороны — архитектурная ошибка.
- Лечение: общий кусок вынести в третий модуль,
- либо импортировать внутри функции (отложить).

круг и развязка

```
# rates.py
from .report import build   # (1)
# report.py
from .rates import oil_rate  # (2)
# ImportError: cannot import name

# решение: общее - в base.py,
# оба тянут из него односторонне:
rates.py  ->  base.py  <-  report.py
```

Не то окружение

- Симптом: «pandas ставил, а import не работает».
- Причина: pip выполнен в системном Python,
- а скрипт запущен из .venv (или наоборот).
- pip и python РАЗНЫЕ интерпретаторы —
- pip ставит «себе», а не тому, кем запускаете.
- Правило: python -m pip install ...
- — pip гарантированно того же интерпретатора.
- Проверка: where pip, where python —
- обе команды из одной ли папки .venv?

терминал

```
(.venv) PS> where.exe python
C:\proj\.venv\Scripts\python.exe
C:\Python312\python.exe

(.venv) PS> python -m pip install pandas
# гарантированно в .venv

# без активации можно напрямую:
PS> .venv\Scripts\python -m pip -V
pip 24.2 from ...\.venv\...
```

Кэш, звёздочка, перезапуск

- 1) Правите модуль, а эффект «не наступает»:
- модуль в кэше sys.modules — перезапустите
- скрипт / ядро Jupyter (Restart Kernel).
- 2) from m import * — залитие имён и сюрпризы;
- точечный импорт лечит.
- 3) Свой rates.py ЗАТЕНЯЕТ чужой пакет rates:
- не называйте файлы как популярные библиотеки
- (email.py, random.py, json.py — нельзя!).
- Три привычки: перезапуск после правок,
- никаких звёздочек, аккуратные имена файлов.

```
классика жанра
# в папке проекта лежит json.py
>>> import json
>>> json.dumps({'a': 1})
AttributeError: module 'json' has no
attribute 'dumps'   # взяли ВАШ файл!

# в Jupyter после правок rates.py:
>>> import importlib, rates
>>> importlib.reload(rates)   # или Restart
```

ЗАКРЕПИМ

Чек-лист по модульности

`import m / from m import x` умолчание — первое; звёздочка запрещена.

`__name__ == '__main__'` код запуска отделён от кода библиотеки.

`sys.path` порядок поиска: скрипт → stdlib → site-packages.

`__init__.py` маркер пакета и его публичный API.

`requirements.txt` контракт зависимостей; freeze — снимок.

`python -m pip` pip того же интерпретатора, что и запуск.

`.venv` папка проекта; в git не идёт, requirements — да.

`ModuleNotFoundError` три вопроса: имя? откуда? тот ли Python?

Итоги лекции

1. Модуль — файл .py со своим пространством имён; import выполняет его один раз.
2. Формы: import m, from m import x, import m as p; звёздочка — запрет.
3. if __name__ == '__main__' — двойная жизнь: библиотека и скрипт.
4. Пакет — папка с __init__.py; точка в импорте спускается в папку.
5. pip ставит пакеты из PyPI в site-packages; версии фиксируем (==, ~=).
6. requirements.txt — воспроизводимость: pip freeze и pip install -r.
7. venv даёт каждому проекту свой интерпретатор и свои пакеты.
8. Диагностика: sys.path, pip list, where python; круги импортов развязывать архитектурой.

Вопрос 1 из 6

Чем import m отличается от from m import x и почему from m import * — плохо?

Вопрос 2 из 6

Что делает if __name__ == '__main__' и когда блок выполнится?

Вопрос 3 из 6

Зачем пакету __init__.py и как он задаёт публичный API?

Вопрос 4 из 6

Что такое requirements.txt, как его создать и как по нему восстановить окружение?

Вопрос 5 из 6

Зачем нужно виртуальное окружение и где окажется pandas после pip install в активном venv?

Вопрос 6 из 6

Получили ModuleNotFoundError — назовите три первых вопроса для диагностики.